

PRACTICAL WEEK11

TASK1: Develop a multithreaded counter application using POSIX threads (pthread_create, pthread_join). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#define NUM_THREADS 4
#define ITERATIONS 100000

int counter = 0;

void *increment_counter(void *arg)
{
    for (int i = 0; i < ITERATIONS; i++)
    {
        counter++;
    }

    return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];

    printf("Starting counter: %d\n", counter);

    /* Create threads */
    for (int i = 0; i < NUM_THREADS; i++)
    {
        if (pthread_create(&threads[i], NULL,
                          increment_counter, NULL) != 0)
        {
            perror("pthread_create");
            return 1;
        }
    }
}
```

```
/* Wait for all threads */
for (int i = 0; i < NUM_THREADS; i++)
{
    pthread_join(threads[i], NULL);
}

printf("Expected counter: %d\n",
       NUM_THREADS * ITERATIONS);

printf("Actual counter: %d\n", counter);

return 0;
}
```

^G Help
^X Exit

^O Write Out
^R Read File

^W Where Is
^_ Replace

^K Cut
^U Paste

^T Execute
^J Justify

```
laasya@laasya: ~/pthread_lab
laasya@laasya:~$ mkdir pthread_lab
laasya@laasya:~$ cd pthread_lab
laasya@laasya:~/pthread_lab$ nano race.c
laasya@laasya:~/pthread_lab$ gcc race.c -o race -pthread
laasya@laasya:~/pthread_lab$ ./race
Starting counter: 0
Expected counter: 400000
Actual counter: 127610
laasya@laasya:~/pthread_lab$ ./race
Starting counter: 0
Expected counter: 400000
Actual counter: 184934
laasya@laasya:~/pthread_lab$ |
```

TASK2: Modify the previous program using mutex locks (`pthread_mutex_lock`, `pthread_mutex_unlock`) and compare the outputs with the race-condition version. Analyze the impact of synchronization.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NUM_THREADS 4
#define ITERATIONS 100000

int counter = 0;

/* Create a mutex */
pthread_mutex_t mutex;

void *increment_counter(void *arg)
{
    for (int i = 0; i < ITERATIONS; i++)
    {
        pthread_mutex_lock(&mutex);

        counter++;

        pthread_mutex_unlock(&mutex);
    }

    return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];

    /* Initialize mutex */
    if (pthread_mutex_init(&mutex, NULL) != 0)
    {
        perror("pthread_mutex_init");
        return 1;
    }
}
```

```

/* Initialize mutex */
if (pthread_mutex_init(&mutex, NULL) != 0)
{
    perror("pthread_mutex_init");
    return 1;
}

printf("Starting counter: %d\n", counter);

/* Create threads */
for (int i = 0; i < NUM_THREADS; i++)
{
    if (pthread_create(&threads[i], NULL,
                      increment_counter, NULL) != 0)
    {
        perror("pthread_create");
        return 1;
    }
}

/* Wait for all threads */
for (int i = 0; i < NUM_THREADS; i++)
{
    pthread_join(threads[i], NULL);
}

printf("Expected counter: %d\n",
       NUM_THREADS * ITERATIONS);

printf("Actual counter: %d\n", counter);

/* Destroy mutex */
pthread_mutex_destroy(&mutex);

return 0;
}

```

```
laasya@laasya: ~/pthread_lab × + v
laasya@laasya:~$ mkdir pthread_lab
laasya@laasya:~$ cd pthread_lab
laasya@laasya:~/pthread_lab$ nano race.c
laasya@laasya:~/pthread_lab$ gcc race.c -o race -pthread
laasya@laasya:~/pthread_lab$ ./race
Starting counter: 0
Expected counter: 400000
Actual counter: 127610
laasya@laasya:~/pthread_lab$ ./race
Starting counter: 0
Expected counter: 400000
Actual counter: 184934
laasya@laasya:~/pthread_lab$ nano mutex.c
laasya@laasya:~/pthread_lab$ gcc mutex.c -o mutex -pthread
laasya@laasya:~/pthread_lab$ ./mutex
Starting counter: 0
Expected counter: 400000
Actual counter: 400000
laasya@laasya:~/pthread_lab$ ./mutex
Starting counter: 0
Expected counter: 400000
Actual counter: 400000
laasya@laasya:~/pthread_lab$ ./mutex
Starting counter: 0
Expected counter: 400000
Actual counter: 400000
laasya@laasya:~/pthread_lab$ ls
mutex mutex.c race race.c
laasya@laasya:~/pthread_lab$ |
```